MSML610: Advanced Machine Learning

# Lesson 09.5: Kalman Filter

**Instructor**: Dr. GP Saggese, gsaggese@umd.edu

**References**:

- AIMA 14: Probabilistic reasoning over time
- https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python

- ***Discrete Bayes Filtering***
- Unidimensional Kalman Filters
- Multivariate Kalman Filters

# Tutorial

- L09_05_01_discrete_bayes_dog.ipynb

# A Discrete Bayesian 1-D Dog

- Dog with sensor
  - Wanders around offices and halls
- 10 positions in hallway with doors, numbered 0 to 9
  - Circular hallway: position 0 follows position 9
  - Door positions: 0, 1, 8
- Sensor reports if dog is in front of door or hall and its movement
  - Sensor may have noise

(Nuvolo)

- Can we find out where the dog is from consecutive measurements?

# You Cannot Trust Either Source Alone

- **Measurements Alone**
  - Sensor says "door"
  - Three door positions: 0, 1, 8
  - Cannot tell from single measurement which door it is
- **Predictions Alone**
  - Dog moves right repeatedly
  - Position drifts over time
  - Uncertainty accumulates; eventually, no idea where the dog is
- Both sources of information (prediction and measurements) have limitations, but together they work
  - **Combine predictions with measurements**
    - Prediction: *"Based on movement, dog should be around position 4"*
    - Measurement: *"Sensor says wall"*
    - Check hallway: Position 4 is a wall
    - Agreement increases confidence
  - **After several cycles**
    - Only one position consistent with entire history
    - High confidence in dog's location

# Predict: Loses Information

- **What it does**
  - *"Dog was at position 3 and moved right 1"*
  - *"So dog is probably at position 4"*
  - *"But could be 3 or 5 due to movement error"*
- **Effect on belief**
  - Shifts distribution to the right
  - Spreads out probability mass
  - Uncertainty increases
- **Analogy**: Blurring a photo
  - Sharp image becomes fuzzy
  - Information is lost

# Update: Gains Information

- **What it does**
  - *"Sensor says door"*
  - *"Boost probability at all door positions (0, 1, 8)"*
  - *"Reduce probability at wall positions"*
- **Effect on belief**
  - Concentrates probability on matching positions
  - Peaks emerge in distribution
  - Uncertainty decreases
- **Analogy**: Sharpening a photo
  - Fuzzy image becomes clearer
  - Information is gained

# Bayesian Update Algorithm

- **Start with prior belief**

$$Pr(\text{position}) = \text{uniform}$$

  - "Dog could be anywhere"

- **Get evidence** (measurement)

$$Pr(\text{door sensor}|\text{position})$$

  - Likelihood of seeing "door" at each position

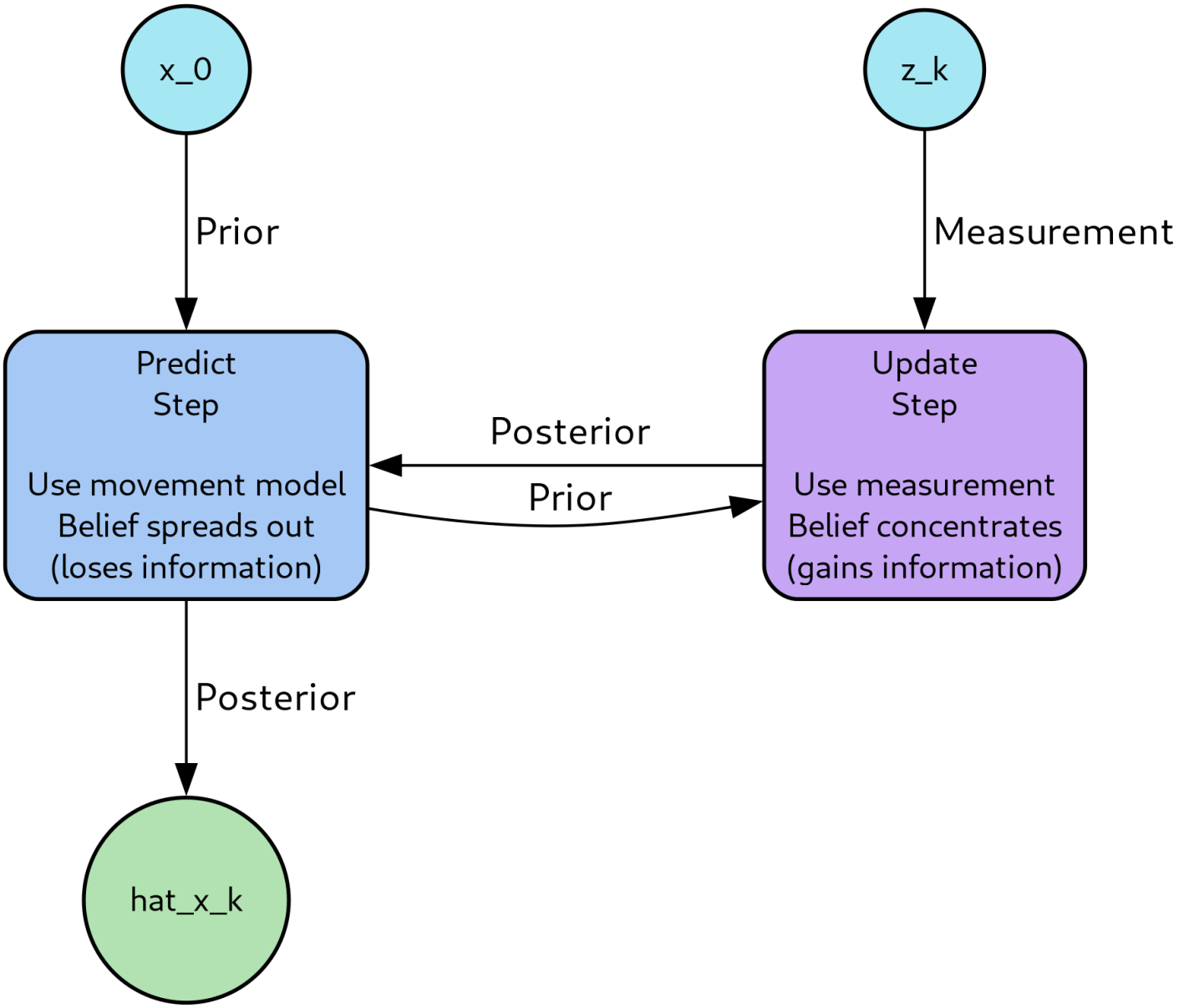
- **Update belief** using Bayes' rule

$$Pr(\text{position}|\text{door sensor}) = \frac{Pr(\text{door sensor}|\text{position}) \cdot Pr(\text{position})}{Pr(\text{door sensor})}$$

- **Posterior becomes next prior**

  - Cycle repeats with each new measurement

# Bayesian Update Algorithm



- Discrete Bayes Filtering
- ***Unidimensional Kalman Filters***
- Multivariate Kalman Filters

# Tutorial

- L09_05_02_univariate_kalman_filter.ipynb

# Kalman Filter: Interpretations

- Extension of $g$–$h$ filters
  - $g$–$h$ uses fixed gains $g, h$ for position and velocity
  - Kalman filter computes adaptive gain $K_t$ from covariance $P_t$
  - Handles multidimensional states and general linear systems
- **State space model**
  - $x_t = Ax_{t-1} + w_t$
  - $y_t = Cx_t + v_t$
  - All quantities are Gaussian with given mean and covariances
- **Bayesian interpretation**
  - Prior and likelihood Gaussian $\Rightarrow$ posterior Gaussian
  - Recursively updates mean and covariance
  - Example: radar tracking of aircraft
- **Orthogonal projection interpretation**
  - Estimate is projection onto span of observations
  - $K_t$ minimizes $\mathbb{E}[\|x_t - \hat{x}_t\|^2]$
  - Error orthogonal to measurement space (optimal)

# Kalman Filter: Bayesian Interpretation

- **Kalman filter** can be interpreted as based on Bayes theorem and Gaussians
- The **Bayes theorem** states:

$$\text{posterior} = \text{normalized}(\text{prior} \times \text{likelihood})$$

- If the *prior* and the *likelihood* are Gaussian the *posterior* is also Gaussian (conjugate prior)
  - Everything can be represented as Gaussians $\rightarrow$ encode PDF in terms of mean and std dev
  - Updating belief is equivalent to sum and multiplication of Gaussians
- **Algorithm**

```
# Create prior (using current estimate and system model).
prior = predict(x, process_model)

# Create likelihood (using measurement).
likelihood = gaussian(z, sensor_var)

# Update belief using prior and likelihood
posterior = update(prior, likelihood)
```

# Sum of Gaussians

- Given two independent Gaussians $X$ and $Y$

$$X \sim Normal(\mu_1, \sigma_1^2)$$
$$Y \sim Normal(\mu_2, \sigma_2^2)$$

- The sum $Z = X + Y$ is $Normal(\mu, \sigma^2)$ with:

$$\begin{cases} \mu = \mu_1 + \mu_2 \\ \sigma^2 = \sigma_1^2 + \sigma_2^2 \end{cases}$$

- **Interpretation:**
  - The mean is the sum of the mean (by linearity)
  - The variance always increases

# Product of Gaussians

- Given two independent Gaussians $X$ and $Y$

$$X \sim Normal(\mu_1, \sigma_1^2)$$
$$Y \sim Normal(\mu_2, \sigma_2^2)$$

- The product $Z = X \cdot Y$ is $Normal(\mu, \sigma^2)$ with:

$$\begin{cases} \mu = \dfrac{\mu_1 \sigma_2^2 + \mu_2 \sigma_1^2}{\sigma_1^2 + \sigma_2^2} \\ \sigma^2 = \dfrac{\sigma_1^2 \sigma_2^2}{\sigma_1^2 + \sigma_2^2} \end{cases}$$

- **Interpretation:**
  - Reduce variance by incorporating more information
  - If one Gaussian $X$ is narrower (more accurate), result leans towards $X$
  - If two Gaussians are similar (measures corroborate), result becomes more certain

# Kalman Filter: Classical Representation

- Typical symbols used in Kalman literature:

| Symbol | Meaning |
|---|---|
| $x$ | State |
| $P$ | Variance of state (uncertainty, belief) |
| $f()$ | System model |
| $Q$ | System model error |
| $z$ | Measurement |
| $R$ | Measurement noise |

- **Initialization**
  - Initialize state of filter: $x = x_0$
  - Initialize belief in the state: $P = P_0$

# Kalman Pseudo-Algorithm

- **Predict stage**

```
def predict(posterior, movement):
  # Prior belief about state (mean x, variance P).
  x, P = posterior
  # System model: movement (mean dx) and its uncertainty Q.
  dx, Q = movement
  # Use system model to predict next state: x = f(x)
  x = x + dx
  # Adjust belief to account for uncertainty in prediction.
  P = P + Q
  return gaussian(x, P)
```

# Kalman Pseudo-Algorithm

- **Update stage**

```
def update(prior, measurement):
  # Prior belief about state (mean x, variance P)
  x, P = prior
  # Measurement z and belief about its accuracy R.
  z, R = measurement
  # Compute residual between estimated state and measurement.
  y = z - x
  # Compute Kalman gain K from prediction uncertainty P and
  # measurement noise R.
  K = P / (P + R)
  # Update state estimate.
  x = x + K * y
  # Update belief (uncertainty).
  P = (1 - K) * P
  return gaussian(x, P)
```

- The Bayesian update is not apparent and it's replaced by Kalman Gain

# Kalman Gain

- Assume that:
  - Prior prediction $x \sim Normal(\mu_x, \sigma_x^2)$
  - Measurement $z \sim Normal(\mu_z, \sigma_z^2)$
- The mean of the posterior (using the multiplication expression) is:

$$\mu = \frac{\sigma_x^2 \mu_z + \sigma_z^2 \mu_x}{\sigma_x^2 + \sigma_z^2} = \frac{\sigma_x^2}{\sigma_x^2 + \sigma_z^2}\mu_z + \frac{\sigma_z^2}{\sigma_x^2 + \sigma_z^2}\mu_x = K\mu_z + (1 - K)\mu_x$$

- The Kalman Gain $K = \dfrac{\sigma_x^2}{\sigma_x^2 + \sigma_z^2}$:
  - Is the scaling term that mixes the prediction and the measurement
  - Depends on the ratio of uncertainty of prior and measurement

- Discrete Bayes Filtering
- Unidimensional Kalman Filters
- ***Multivariate Kalman Filters***
  - Modeling
  - Non-Linear Filtering

# Tutorial

- L09_05_03_multivariate_kalman_filter.ipynb

# Multivariate State

- Often the state variable is multivariate, e.g.,
  - E.g., position and velocity of a dog (probably correlated)
  - E.g., height and weight of an adult (correlated)
- **Variance** is a measure of how a population varies, e.g.,
  - Variance $= 0$ means constant
  - Large variance means lots of variation
- **Covariance** are correlated variances
  - E.g., height and weight are generally positively correlated
- **Covariance matrix**
  - The diagonal contains the variance for each variable
  - The off-diagonal elements contain the covariance between $i$ and $j$ variables
  - The covariance matrix is symmetric
- Correlation allows prediction
  - E.g., "as winter comes you predict you will spend more on heating your house"

# Multivariate Gaussian

- The marginal of a multivariate Gaussian is 1-d Gaussian
- Consider a contour plot (i.e., the intersection of a 2-d Gaussian $z = f(x, y)$ with a plane $z = c$)
  - The contour plot is always an ellipses

# Multiplying Two Multivariate Gaussians

- Given two multivariate Gaussians $\sim Normal(\underline{\boldsymbol{\mu}}_i, \underline{\underline{\boldsymbol{\Sigma}}}_i)$
- The product of the Gaussians is still Gaussian $\sim Normal(\underline{\boldsymbol{\mu}}, \underline{\underline{\boldsymbol{\Sigma}}})$

$$\underline{\boldsymbol{\mu}} = \underline{\underline{\boldsymbol{\Sigma}}}_2(\underline{\underline{\boldsymbol{\Sigma}}}_1 + \underline{\underline{\boldsymbol{\Sigma}}}_2)^{-1}\underline{\boldsymbol{\mu}}_1 + \underline{\underline{\boldsymbol{\Sigma}}}_1(\underline{\underline{\boldsymbol{\Sigma}}}_1 + \underline{\underline{\boldsymbol{\Sigma}}}_2)^{-1}\underline{\boldsymbol{\mu}}_2$$
$$\underline{\underline{\boldsymbol{\Sigma}}} = \underline{\underline{\boldsymbol{\Sigma}}}_1(\underline{\underline{\boldsymbol{\Sigma}}}_1 + \underline{\underline{\boldsymbol{\Sigma}}}_2)^{-1}\underline{\underline{\boldsymbol{\Sigma}}}_2$$

- **Note**: this is a generalization of the 1-d case to multivariate

$$\mu = \frac{\mu_1\sigma_2^2 + \mu_2\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$
$$\sigma^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}$$

replacing:

  - $\sigma^2$ with covariance matrix $\underline{\underline{\boldsymbol{\Sigma}}}$
  - Division with matrix inversion

# Core Insight: Power Comes from Correlated Structure

- Real physical systems couple variables through dynamics
  - Position evolves according to $x_{t+1} = x_t + \dot{x}_t \Delta t$
  - A change in $\dot{x}$ necessarily changes future $x$
- Correlation constrains what combinations are plausible
  - Large velocity with small position change is unlikely
  - Bayesian update favors consistent $(x, \dot{x})$ pairs
- No correlation means no information gain
  - If velocity comes from a different aircraft, combining it with position adds nothing
  - Mathematically, zero covariance implies no reduction in uncertainty
- Hidden but correlated variables create extra constraints
  - Velocity is not measured, yet strongly linked to position
  - Each measurement of $x$ indirectly informs $\dot{x}$
- Uncertainty shrinks because we intersect constraints
  - Measurement constrains $x$
  - Dynamics constrain relationship between $x$ and $\dot{x}$
  - Posterior covariance reflects both constraints
  - Multivariate filter exploits structure of motion

# Multivariate Filtering

- Covariance structure helps improve the estimate, e.g.,
  - You know an airplane direction can't change quickly
  - Knowing an approximate value for the velocity helps constrain possible next positions
- State variables can either be:
  - observed variables: directly measured by a sensor
  - hidden variables: inferred from the observed variables
- **Example: airplane**
  - You are tracking a plane moving in a direction (1-d problem)
  - At time 1, you are fairly certain about the position $x = 0$, but you don't know the velocity
    - You plot position and velocity on an x-y plane
    - The covariance matrix between position and velocity is narrow and tall
    - It is narrow on the x-axis since you know that the position is around $x = 0$
    - It is tall on the y-axis because of your lack of knowledge about velocity
  - After 1 sec, you get a position update of $x = 5$
    - You can infer that the velocity is $5/s$

# Notation

- A Bayesian notation $a|b$ means *"a given the evidence of b"*
  - The prior is $\underline{\hat{\boldsymbol{x}}}_{t|t-1}$, since you know only the information at time $t-1$, i.e., the previous state
  - The posterior is $\underline{\hat{\boldsymbol{x}}}_{t|t}$, since you know all the information at time $t$, i.e., the measurement
- **A simpler notation:**
  - Indicate the "prior" version of the variables (i.e., after the system update) with an overline (E.g., $\overline{x}$, $\underline{\overline{\boldsymbol{x}}}$, $\underline{\overline{\boldsymbol{X}}}$)
  - Omit the indices $t+1$ and $t$ and use an assignment notation (representing "update in place" of a variable):

    $$x = x + 1$$

    instead of the mathematical notation using a different variable for each time step:

    $$x_{t+1} = x_t + 1$$

  - With this notation:
    * The prior is $\overline{x} = \hat{x}_{t|t-1}$
    * The posterior is $x = \hat{x}_{t|t}$

# Multivariate Kalman Filter

- With the previous notation:
  - State update: $\underline{\overline{x}} = \underline{\underline{F}}\underline{x} + \underline{\underline{B}}\underline{u}$
  - State uncertainty: $\underline{\underline{\overline{P}}} = \underline{\underline{F}}\underline{\underline{P}}\underline{\underline{F}}^T + \underline{\underline{Q}}$
  - Residual: $\underline{y} = \underline{z} - \underline{\underline{H}}\underline{\overline{x}}$
  - Kalman gain: $\underline{\underline{K}} = \underline{\underline{\overline{P}}}\underline{\underline{H}}^T(\underline{\underline{H}}\underline{\underline{\overline{P}}}\underline{\underline{H}}^T + \underline{\underline{R}})^{-1}$
  - Updated state: $\underline{x} = \underline{\overline{x}} + \underline{\underline{K}}\underline{y}$
  - Update state uncertainty: $\underline{\underline{P}} = (\underline{\underline{I}} - \underline{\underline{K}}\underline{\underline{H}})\underline{\underline{\overline{P}}}$
- Where
  - $\underline{x}$ and $\underline{\underline{P}}$ are the state mean and covariance
  - $\underline{\underline{F}}$ is the state transition function
  - $\underline{\underline{Q}}$ is the system error (i.e., the noise in the model assessment)
  - $\underline{\underline{B}}$ and $\underline{u}$ model the control inputs to the system
  - $\underline{\underline{H}}$ is the measurement function
  - $\underline{z}$ and $\underline{\underline{R}}$ are the measurement mean and covariance
  - $\underline{y}$ is the residual
  - $\underline{\underline{K}}$ is the Kalman gain
- Use the system model to predict the next state
  - When we multiply $\underline{\underline{F}}$ to $\underline{x}$ we get the prior (i.e., the state before seeing any measurement)

# From Univariate to Multivariate Kalman Filter

- Let's compare

| Definition | Univariate (Bayesian) | Univariate (Kalman) | Multivariate (Kalman) |
|---|---|---|---|
| State update | $\overline{\mu} = \mu + \mu_f$ | $\overline{x} = x + dx$ | $\underline{\overline{\boldsymbol{x}}} = \underline{\underline{\boldsymbol{F}}}\underline{\boldsymbol{x}} + \underline{\underline{\boldsymbol{B}}}\underline{\boldsymbol{u}}$ |
| State uncertainty | $\overline{\sigma}^2 = \sigma^2 + \sigma_f^2$ | $\overline{P} = P + Q$ | $\underline{\underline{\overline{\boldsymbol{P}}}} = \underline{\underline{\boldsymbol{F}}}\underline{\underline{\boldsymbol{P}}}\underline{\underline{\boldsymbol{F}}}^T + \underline{\underline{\boldsymbol{Q}}}$ |
| Residual | | $y = z - \overline{x}$ | $\underline{\boldsymbol{y}} = \underline{\boldsymbol{z}} - \underline{\underline{\boldsymbol{H}}}\underline{\overline{\boldsymbol{x}}}$ |
| Kalman gain | | $K = \frac{\overline{P}}{\overline{P}+R}$ | $\underline{\underline{\boldsymbol{K}}} = \underline{\underline{\overline{\boldsymbol{P}}}}\underline{\underline{\boldsymbol{H}}}^T(\underline{\underline{\boldsymbol{H}}}\underline{\underline{\overline{\boldsymbol{P}}}}\underline{\underline{\boldsymbol{H}}}^T + \underline{\underline{\boldsymbol{R}}})^{-1}$ |
| Updated state | $\hat{\mu} = \frac{\overline{\sigma}^2\mu_z + \sigma_z^2\overline{\mu}}{\overline{\sigma}^2 + \sigma_z^2}$ | $x = \overline{x} + Ky$ | $\underline{\boldsymbol{x}} = \underline{\overline{\boldsymbol{x}}} + \underline{\underline{\boldsymbol{K}}}\underline{\boldsymbol{y}}$ |
| Upd. state uncertainty | $\sigma^2 = \frac{\overline{\sigma}^2\sigma_z^2}{\overline{\sigma}^2 + \sigma_z^2}$ | $P = (1-K)\overline{P}$ | $\underline{\underline{\boldsymbol{P}}} = (\underline{\underline{\boldsymbol{I}}} - \underline{\underline{\boldsymbol{K}}}\underline{\underline{\boldsymbol{H}}})\underline{\underline{\overline{\boldsymbol{P}}}}$ |

# Designing a Kalman filter

- The designer of the model needs to design:
  - The form of the state $\underline{\boldsymbol{x}}$ and $\underline{\underline{\boldsymbol{P}}}$
  - The system model $\underline{\underline{\boldsymbol{F}}}$ and $\underline{\underline{\boldsymbol{Q}}}$
  - The measurement $\underline{\boldsymbol{z}}$ and $\underline{\underline{\boldsymbol{R}}}$
  - The measurement function $\underline{\underline{\boldsymbol{H}}}$
  - The control inputs $\underline{\underline{\boldsymbol{B}}}$ and $\underline{\boldsymbol{u}}$ if there are control inputs

# Algorithm

**Initialization** - Initialize the state of the filter - Initialize our belief in the state

**Predict** - Use process model to predict state at the next time step - Adjust belief to account for the uncertainty in prediction

**Update** - Get a measurement and associated belief about its accuracy - Compute residual between estimated state and measurement - Compute scaling factor based on whether the measurement or prediction is more accurate - Set state between the prediction and measurement based on scaling factor - Update belief in the state based on how certain we are in the measurement

- Discrete Bayes Filtering
- Unidimensional Kalman Filters
- Multivariate Kalman Filters
  - ***Modeling***
  - Non-Linear Filtering

# State Space Systems

- A dynamic system is a system whose state (position, temperature) evolves over time
- Many dynamic system can be described as differential equations, since calculus is the math of changing values
- It is possible to convert a set of higher order differential equations into a set of first-order differential equations

$$\dot{\underline{\boldsymbol{x}}} = \underline{\underline{\boldsymbol{A}}}\underline{\boldsymbol{x}} + \underline{\underline{\boldsymbol{B}}}\underline{\boldsymbol{u}} + \underline{\boldsymbol{w}}$$

  where

  - $\underline{\underline{\boldsymbol{A}}}$ is the system dynamics matrix
  - $u$ are control inputs (e.g., pressing accelerator)
  - $\underline{\boldsymbol{w}}$ is noise

- We are interested in finding a (recursive) equation that finds the value of $\underline{\boldsymbol{x}}_k = \underline{\boldsymbol{x}}(t_k)$ from the previous (discrete) state $\underline{\boldsymbol{x}}_{k-1}$

$$\underline{\boldsymbol{x}}_k = \underline{\underline{\boldsymbol{F}}}\underline{\boldsymbol{x}}_{k-1} + \underline{\underline{\boldsymbol{B}}}_k\underline{\boldsymbol{u}}_k$$

  where

  - $\boldsymbol{F}$ is the state transition matrix (aka "fundamental matrix")

# Designing the Process Noise Matrix

- Designing the matrix $\underline{\underline{\boldsymbol{Q}}}$ is very difficult
  - When you model a given process, you ignore certain effects and then you need to estimate the error
- E.g., model a thrown baseball
  - There are unknown factors (e.g., ball rotation, spin decay, drag, effects of wind and air density)
- If $\underline{\underline{\boldsymbol{Q}}}$ is too small, the filter is overconfident in the prediction model
- If $\underline{\underline{\boldsymbol{Q}}}$ is too large, the filter is unduly influenced by the noise in the measurements

- Discrete Bayes Filtering
- Unidimensional Kalman Filters
- Multivariate Kalman Filters
  - Modeling
  - ***Non-Linear Filtering***

# Optimality

- The Kalman filter is optimal (in a least square sense) under **assumptions** that
  - Everything is linear
  - Process and sensor noise is Gaussian

# The World Is Non-Linear

- The Kalman filter uses linear equations and can only handle linear problems
  - The Kalman filter is a mathematical model of the world
  - The output is only as accurate as the model of the world
- **The world is non-linear:**
  - System model can be non-linear:
    - Many physical systems are described by non-linear differential equations
    - E.g., a ball flying through air is affected by drag
  - Measurements can be non-linear:
    - To measure the height on a plane, you can measure the distance of the plane from the radar. Given the Pythagorean theorem, you get:

$$x = \sqrt{\text{dist}^2 - \text{height}^2}$$

  - A Gaussian through a almost linear f(x) becomes quickly non-Gaussian (bimodal, skewed, etc)
- Rarely does a physical system have equations that can be solved analytically

# Nonlinear Kalman Filtering Overview

- Goal: estimate hidden state $x_t$ of nonlinear dynamical system
  - State dynamics: $x_t = f(x_{t-1}, u_{t-1}) + w_t$
  - Measurement model: $y_t = h(x_t) + v_t$ where
  - $f(\cdot)$ and $h(\cdot)$ nonlinear
- Noise often assumed Gaussian:
  - $w_t \sim \mathcal{N}(0, \underline{\underline{\boldsymbol{Q}}})$
  - $v_t \sim \mathcal{N}(0, \underline{\underline{\boldsymbol{R}}})$
- Exact Bayesian filter requires propagating full probability density
  - Generally intractable
- Nonlinear Kalman filters approximate the posterior
  - Maintain tractable representation of mean and covariance
- Different approaches
  - Extended Kalman Filter (EKF)
  - Unscented Kalman Filter (UKF)
  - Particle Filter

# Extended Kalman Filter EKF

- **Core idea**: linearize nonlinear functions around current estimate
  - First order Taylor expansion of $f$ and $h$
- Replace nonlinear system by local linear model
  - Jacobians: $F_t = \frac{\partial f}{\partial x}$, $H_t = \frac{\partial h}{\partial x}$
- Apply standard Kalman filter equations to linearized model
- Intuition
  - Approximate nonlinear curve by tangent line at estimate
  - Assume distribution remains approximately Gaussian
- Pros
  - Simple and computationally efficient
  - Use the linear Kalman machinery
- Cons
  - Accuracy depends on quality of linearization
  - Can diverge for strong nonlinearities

# Unscented Kalman Filter UKF

- **Core idea**: avoid linearization, approximate distribution instead
- Represent Gaussian by deterministic sigma points
  - Choose $2n + 1$ points capturing mean and covariance
- Propagate sigma points through nonlinear functions
  - $x_t^{(i)} = f(x_{t-1}^{(i)})$
- Recompute mean and covariance from transformed points
- Intuition
  - More accurate to approximate distribution than function
  - Captures second order nonlinear effects
- Pros
  - No Jacobians required
  - Higher accuracy than EKF for strong nonlinearities
  - It is superior to EKF in almost every way
- Cons
  - Slightly higher computational cost

# Unscented Transform

- Unscented transform estimates the result of applying a non-linear transformation to a probability distribution characterized by a finite number of moments (e.g., mean and covariance)
  - E.g., compute the non-linear transform of a distribution, given mean and covariance estimate
  - Called "unscented" since "it doesn't stink."
- **Intuition**
  - Given a PDF $C$ with mean $\underline{\boldsymbol{\mu}}$ and covariance $\underline{\underline{\boldsymbol{\Sigma}}}$
  - Encode mean and covariance in a set of points (sigma points) that represent a discrete PMF $D$ with the same mean $\underline{\boldsymbol{\mu}}$ and covariance $\underline{\underline{\boldsymbol{\Sigma}}}$
  - Propagate the discrete PMF $D$ by applying the non-linear function $\phi$ to each point of the PMF
  - The mean and covariance of $\phi(D)$ approximate the mean and covariance of $\phi(C)$

# Unscented Transform: 1D Case

- The idea is that we need 3 sigma points for a 1-d Gaussian
  - One point for the mean
  - Two points around the mean
- Each point has a weight

# Unscented Transform: Sigma Points

- Consider a distribution $F$ and a non-linearity $\phi$
- There are algorithms to generate points and weights (given the mean and covariance of $F$) to evaluate mean and covariance of $F$ transformed through $\phi$
- In $n$ dimensions, we need $2n+1$ points $\underline{\boldsymbol{x}}_i$ and weights $w_i^m, w_i^c$

$$\sum_i w_i^m = 1$$

$$\sum_i w_i^c = 1$$

$$\mu(\phi) = \sum_i w_i^m \phi(\underline{\boldsymbol{x}}_i)$$

$$\Sigma(\phi) = \sum_i w_i^c (\phi(\underline{\boldsymbol{x}}_i) - \mu(\phi))(\phi(\underline{\boldsymbol{x}}_i) - \mu(\phi))^T$$

  - Note that selecting the sigma points has not a single solution

# Particle Filter PF

- **Core idea**: represent posterior by weighted samples
  - $\{x_t^{(i)}, w_t^{(i)}\}_{i=1}^N$
- Sequential Monte Carlo approximation of Bayesian filter
- Steps
  - Sample from proposal distribution
  - Weight by likelihood $p(y_t|x_t^{(i)})$
  - Resample to avoid weight degeneracy
- Intuition
  - Approximate arbitrary distribution, not restricted to Gaussian
  - Works for multimodal and highly nonlinear systems
- Pros
  - Very flexible
  - Handles non-Gaussian noise
- Cons
  - Computationally expensive
  - Particle degeneracy problem

# Intuition of Sampling Techniques

- Assume you have a distribution $X$ and a non-linearity $\phi$
- For every measurement:
  - Generate many points from $X$
  - Pass them through the non-linear function $\phi$
  - Approximate the result (e.g., compute mean and variance)
- **Problem**:
  - "How many points are needed to build an accurate output distribution"?
  - Even if $n$=500,000 points are enough for 1 dimension, for $k$ dimensions you might need $n^k$ points (curse of dimensionality)

# Generic particle filter algorithm

1. Randomly generate particle

- Particles have all state variable that needs to be estimated (e.g., position, velocity)
- Each particle has a weight representing the probability that it represents the actual state of the system

2. Predict next state of the particles
3. Update weighting

- Update the weighting of the particles based on the measurements
- Particles that match closely the measurements are weighted higher

4. Resample

- Discard highly improbable particle

5. Compute estimate

- Compute weighted mean and covariance of the particles to get an estimate of the state and uncertainty

# Degeneracy Problem in Sequential Importance Sampling (SIS)

- Intuition: weights collapse onto very few particles
  - Most particles receive extremely small weights after measurement update
  - Only a handful meaningfully contribute to the state estimate
- Why this happens
  - Initial particles are uniformly distributed
  - Measurement likelihood sharply favors particles near the true state
- Particles with near-zero weight
  - Contribute almost nothing to expectations
  - Do not help represent the true distribution
- Geometric intuition
  - Posterior mass concentrated in small region of state space
  - Most particles lie in low-probability regions
- Consequence
  - Effective sample size becomes very small
  - Example: 5000 particles but only 3 with significant weight
- Interpretation
  - The approximation of the posterior is poor despite large $N$

# Resampling: Core Idea

- Goal: focus computational effort on high-probability regions
- Strategy
  - Discard particles with very small weights
  - Duplicate particles with high weights
- Effect
  - New particle set better approximates posterior
  - Majority of particles lie in meaningful regions
- Role of prediction noise
  - Slight dispersion after duplication
  - Prevents particles from collapsing to identical copies